

Modern, solid

and testable ABAP Code

Agenda

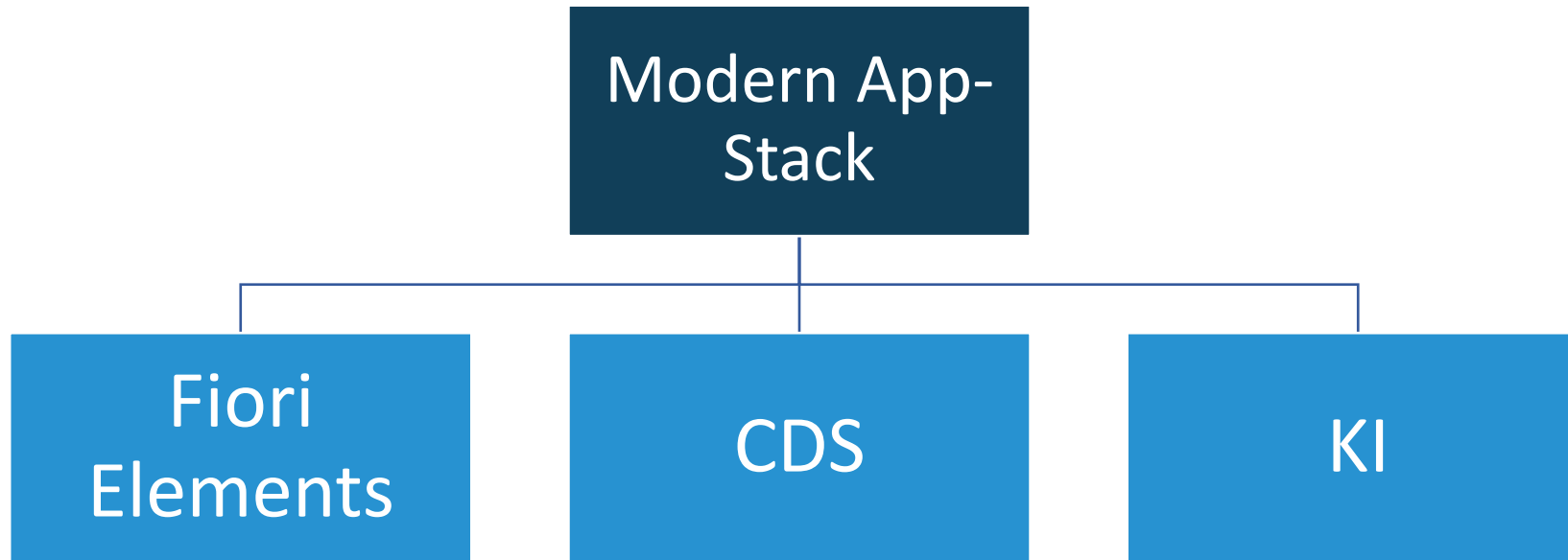
1. Introduction
2. Pattern
3. ABAP Unit
4. Tips
5. Conclusion

WHY
NOT?

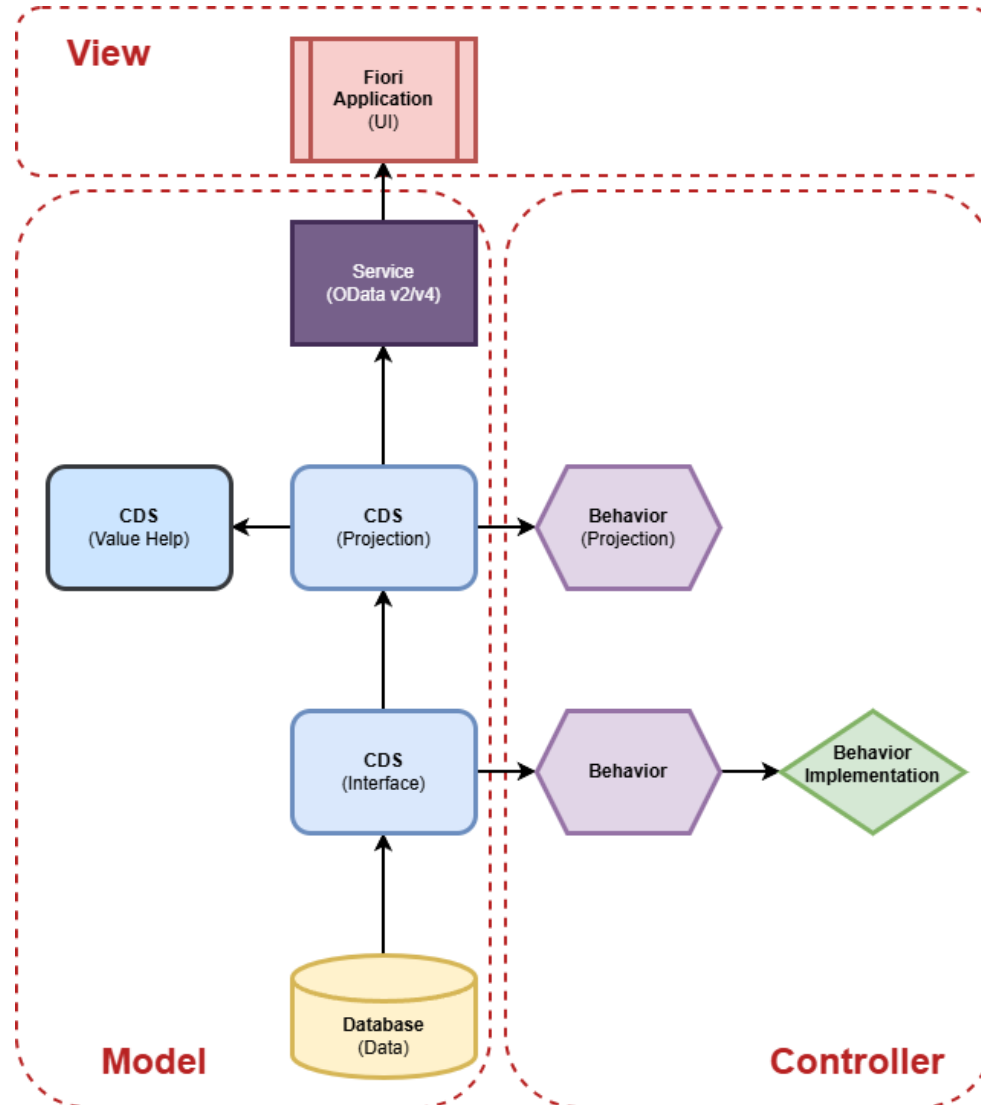
Introduction

Introduction

Why still use ABAP?



Introduction



ABAP and
ABAP Unit

Clean ABAP

Structure and
encapsulation

Introduction

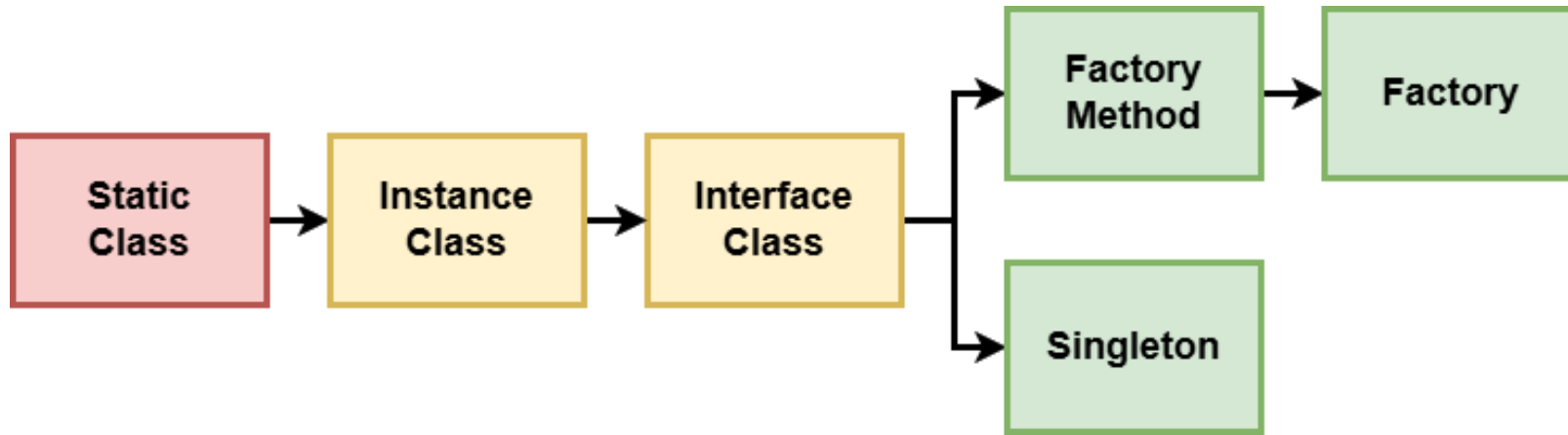
What can you expect?





Pattern

Pattern



Structure

- Generate and save timestamps
- Access via an attribute

Pattern

Examples

Muster	Example
Static	CL_ABAP_CONTEXT_INFO
Instance	CL_ABAP_GZIP_BINARY_STREAM
Interface	CL_API_JT_CHECK_BASE
Factory Method	CL_SXML_STRING_READER
Factory	CL_IAM_BUSINESS_USER_FACTORY
Singleton	CL_SWF_TST_CONTAINER_SINGLETON

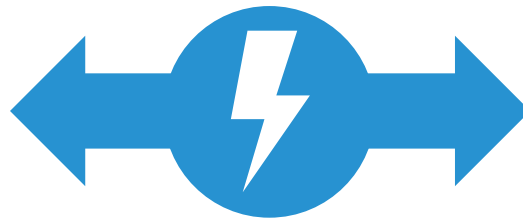
ABAP Unit

ABAP Unit



No Test

- Manual review of various code components
- Risk of untested code
- Fear of changing existing logic

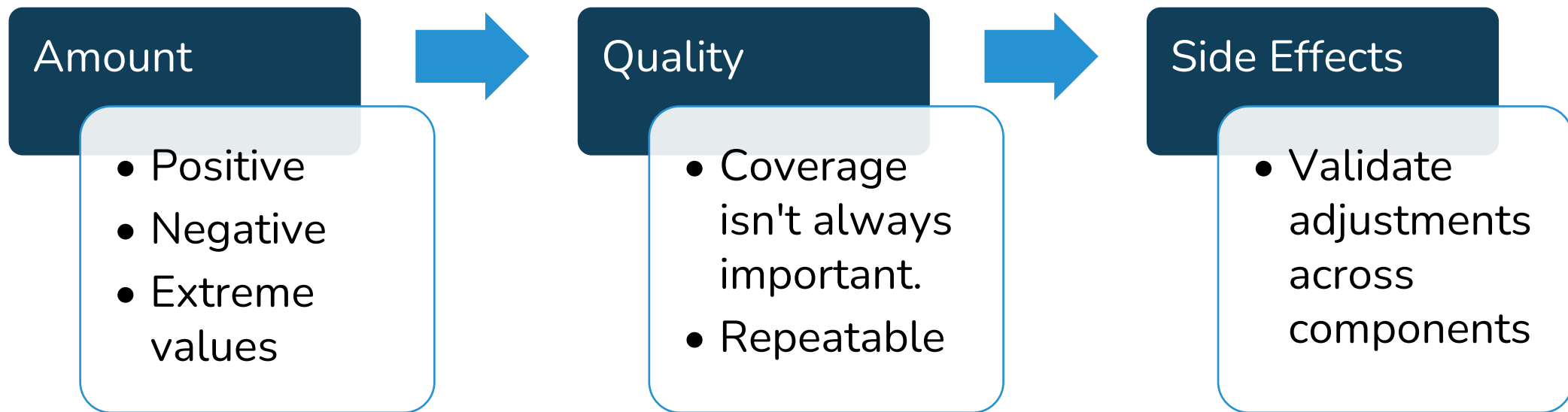


Unit Test

- Validation against results directly during development
- Safety net for changes
- Tracking of special cases
- Documentation of functionality



ABAP Unit



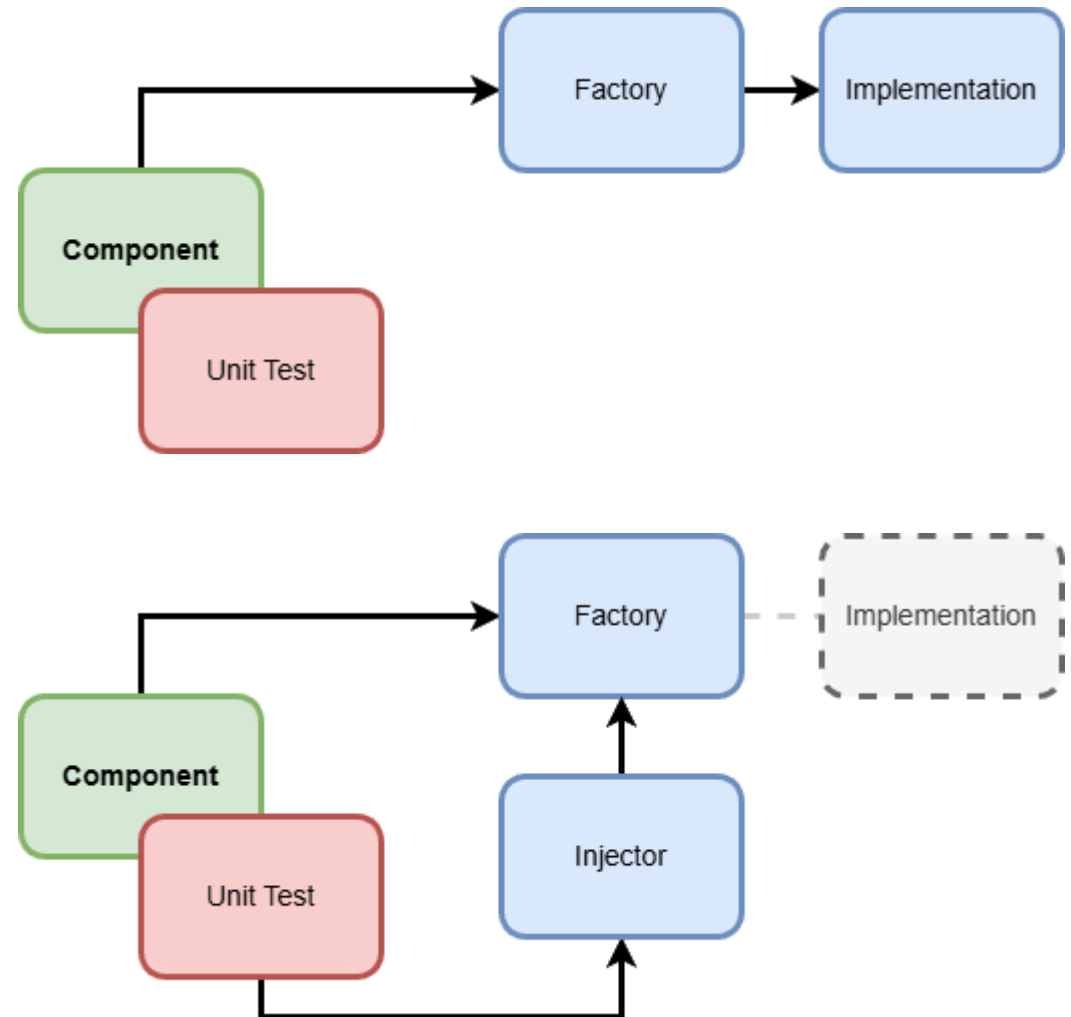
ABAP Unit

Scenario

- Using the Factory
- Creating a Unit Test on the Using Component

Injector

- Option to replace the implementation
- No code modification required





Tips

Tips

Tools and Information

Avoid obsolete language elements

[Clean ABAP > Content > Language > This section](#)

When upgrading your ABAP version, make sure to check for obsolete language elements and refrain from using them.

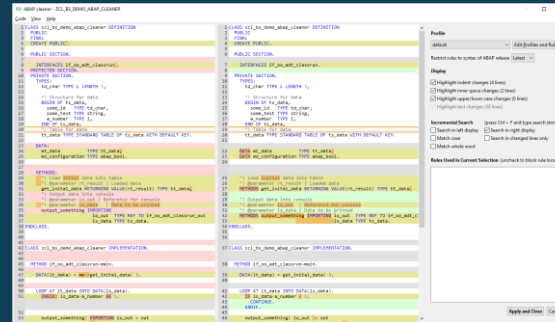
For example, the @-escaped "host" variables in the following statement make a little clearer what's a program variable and what's a column in the database,

```
SELECT *  
FROM spfl1  
WHERE carrid = @carrid AND  
connid = @connid  
INTO TABLE @itab.
```

as compared to the [obsolete unescaped form](#)

```
SELECT *  
FROM spfl1  
WHERE carrid = carrid AND  
connid = connid  
INTO TABLE itab.
```

Clean ABAP



ABAP Cleaner

The screenshot shows the Code Pal for ABAP tool interface. It displays a list of code snippets with various annotations and a search bar. The tool provides various annotations and a search bar to help users identify and clean up obsolete language elements.

Description	Line Number	Object Type	Check
Number of attributes must be lower than 121 (13+ = 12)	1	CLAS	Number of Attributes
Prefer NEW to CREATE OBJECT	2 (CREATE)	CLAS	Prefer New to Create Object
CALL METHOD statement should not be used	4 (DO_IT_AGAIN)	CLAS	CALL METHOD Usage
CK_ROOT should not be used	5 (GET_USER)	CLAS	CK_ROOT usage
1 public methods without interface	6	CLAS	Interface Missing
Consider calling the RETURNING parameter RESULT	14 (ZCL_CLEAN_UP_CLASS)	CLAS	Returning Name
Method DO_IT has a misleading name for boolean return type	16 (ZCL_CLEAN_UP_CLASS)	CLAS	Method Name Misleading for Boolean Return
Split methods instead of adding OPTIONAL parameter	2 (ZCL_EXTRACT_METHOD)	CLAS	Optional Parameters
Magic Number Violation - 255 is a Magic Number!	2 (SET_VALUE)	CLAS	Magic Number Usage
Magic Number Violation - 255 is a Magic Number!	3 (ADD_VALUE)	CLAS	Magic Number Usage
Consider calling the RETURNING parameter RESULT	8 (ZCL_MAGIC_NUMBER_UP_CLASS)	CLAS	Returning Name

Code Pal for ABAP

Tips

My IDE Actions (MIA)

- Use of IDE actions to create artifacts
- Clear rules and adherence to guidelines
- Automation of effort, even without AI

The screenshot displays two overlapping IDE windows. The top window, titled 'Create new class', contains the following fields:

- Package: * ZBS_DEMO_GENERATED
- Transport: * 759
- Prefix: * Z
- Name: * MIA_RUN
- Description: * MIA Runner
- Class: * ZCL_MIA_RUN
- Interface: ZIF_MIA_RUN
- Factory: ZCL_MIA_RUN_
- Injector:

The bottom window, also titled 'Create new class', shows the 'HTML Action Result' for the same operation. It displays the 'Result [759]' and a table of generated artifacts:

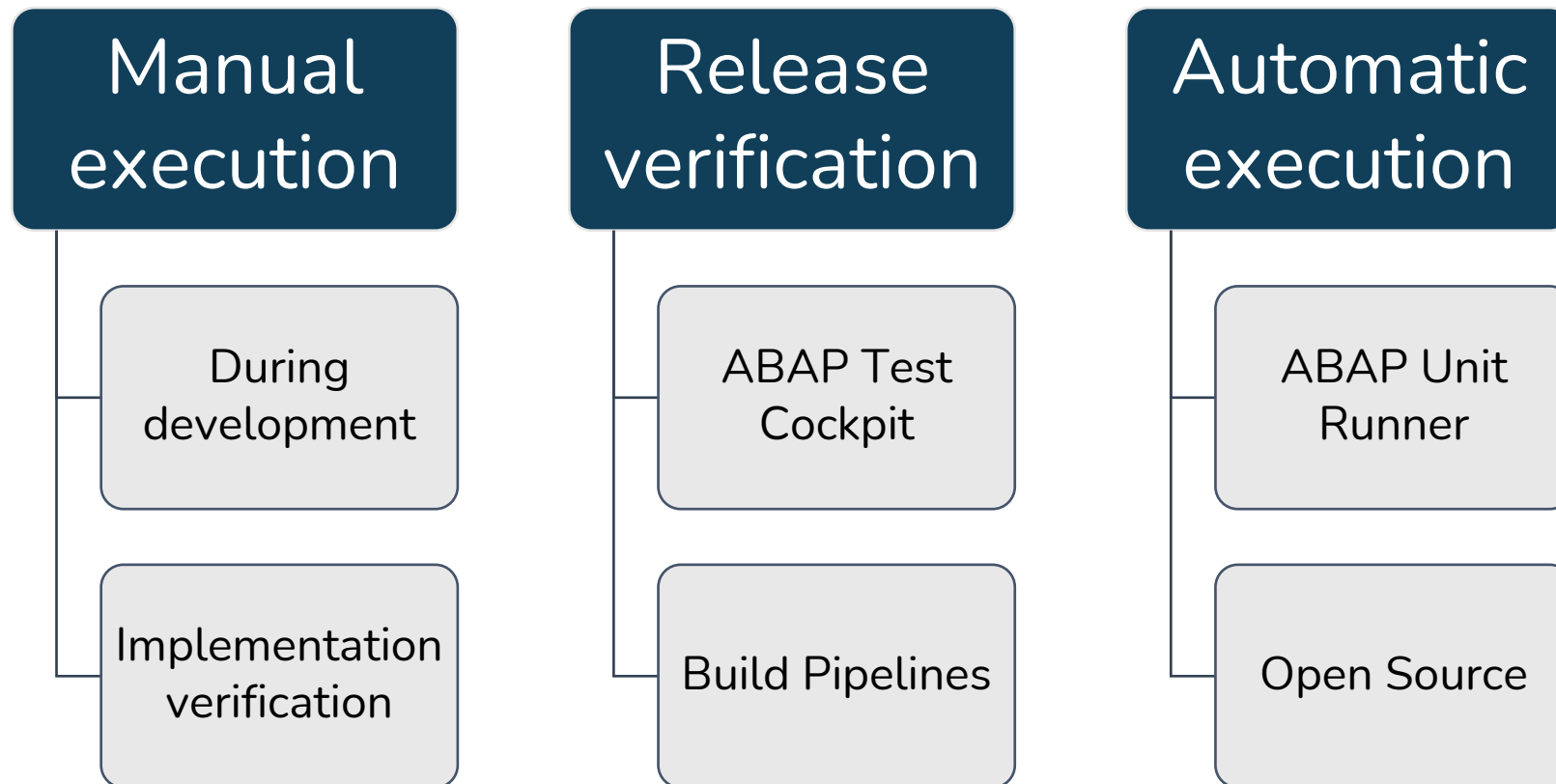
Object	Name
Interface	ZIF_MIA_RUN
Class	ZCL_MIA_RUN
Factory	ZCL_MIA_RUN_FACTORY

Below the table, a tree view shows the 'Generated Objects' for 'ZBS_DEMO_GENERATED (3)':

- Classes (2)
 - > ZCL_MIA_RUN MIA Runner (Class)
 - > ZCL_MIA_RUN_FACTORY MIA Runner (Factory)
- Interfaces (1)
 - > ZIF_MIA_RUN MIA Runner (Interface)

Tipps

Regular automatic tests



Tipps

ABAP Unit Runner

Select Objects for Unit Tests

Select by Package: ☒
 Select by Program: ☐

Select by Package

Packages: 

With Subpackages: ☒

Exclude Selected Objects...: ☐

ABAP Unit

Overview

Variant:
 System:
 Client:
 Started by:
 Started on: 04.01.2024
 Started at: 16:21:41 (CET)
 Finished at: 16:21:49 (CET)

• Fatal Failures: 1
 • Critical Failures: 7

Method Statistics

Packages	Programs	Test Classes	Test Methods	Passed	With Fatal	With Critical	With Tolerable	Skipped/No Exec.
6	12	12	46	30	1	6	0	1

Failures

Level	Test Class	Test Method	Description	Object Name	Type	Package
▲	LTC_LOOS	n.a.	Exception Error <CXX_CA_LOG_ERROR>		CLAS	
▲	LCL_TST_DISTMODEL	ADD_DATA_TEST_MODEL	Critical Assertion Error: 'Failed to update model'		CLAS	
▲	LCL_TST_DISTMODEL	DELETE_DATA_TEST_MODEL	Critical Assertion Error: 'Failed to update model'		CLAS	
▲	LCL_TST_DISTMODEL	READ_DATA_VALID_MODEL	Critical Assertion Error: 'Failed to read model'		CLAS	
▲	LCL_TST_REMOTE	READ_DEST_OK	Critical Assertion Error: 'RFC destination is not valid'		CLAS	
▲	LCL_TST_REMOTE	READ_DEST_OK_SUB	Critical Assertion Error: 'RFC destination is not valid'		CLAS	
▲	LTC_EXTRACTION	n.a.	Runtime Error <PFS>		CLAS	
▲	LTC_TABLE_TEST	FILL_SELECTION	Critical Assertion Error: 'Implement your first test here'		CLAS	

Packages

On-Premise | Private Cloud

Destinations

Destination:
 Communication Arrangement:

Settings

General

Title: AUnit Run
 Wait (Cycles): 60
 Wait (Seconds): 1

Scope

Own: ☒
 Foreign: ☒

Duration

Short: ☒
 Medium: ☐
 Long: ☐

Risk Level

Harmless: ☒
 Dangerous: ☐
 Critical: ☐

Selection

Packages:
 Classes:

Notification

Send mail: ☐
 Only send, when error: ☐
 Sender:
 Receiver:

E-Mail Settings

General Informations

Title	System	Client	Execution User	Run Time	Timestamp	Failures	Errors	Skipped	Asserts	Tests
Testrun This		100		5.120	2025-02-06T19:05:24Z	5	1	0	5	47

Package: ZBS_DEMO_TESTS

ZCL_BS_DEMO_API_OUT [LTC_READ_ENTITY]

Tests	Failures	Errors	Skipped	Asserts	Timestamp	Time
3	1	1	0	1	2025-02-06T19:05:25Z	0.249

Method **Time** **Asserts**

CREATE_FAIL	0.186	0
READ_ENTITY	0.035	0
READ_LIST	0.027	1

ZCL_BS_DEMO_CDS [LTC_ZBS_C_COMPANYCODEOUT]

Tests	Failures	Errors	Skipped	Asserts	Timestamp	Time
1	0	0	0	0	2025-02-06T19:05:28Z	0.215

Open Source Project (Cloud)

Conclusion



Conclusion



Importance

- With RAP and AI, the topic remains important
- Good design remains the responsibility of the developer



Pattern

- Using the correct patterns and names
- Creating testable code



Testing

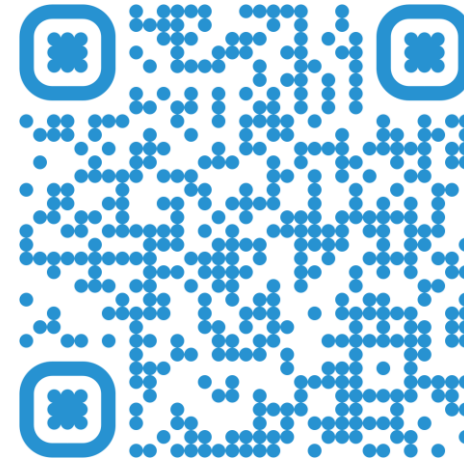
- Unit testing as a minimum standard
- Regular test runs for verification
- Rapid development



Thanks



Download



LinkedIn